Connecting BAYCK ELRS Receiver to ESP32 Using PlatformIO

Instructions for Connecting BAYCK ELRS Receiver to ESP32

Here's how you can connect and configure the BAYCK ELRS 915MHz / 2.4GHz Nano ExpressLRS Receiver with an ESP32.

This setup allows the ESP32 to receive control inputs from an ExpressLRS-compatible transmitter, like the Jumper T14 Macarone.

Materials Needed:

1. BAYCK ELRS Nano Receiver (Make sure it's the same frequency as your transmitter, 915MHz or 2.4GHz).
2. ESP32 Board
3. Jumper Wires

Step 1: Wiring the Receiver to the ESP32

1. Power the Receiver:
   - The receiver typically operates on 5V or 3.3V. Check the receiver's specifications.
   - Connect the receiver's VCC pin to the 5V (or 3.3V, depending on specifications) pin on the ESP32.
2. Ground Connection:
   - Connect the GND pin on the receiver to the GND pin on the ESP32.
3. Connect the Signal Pin:
   - Connect the SBUS (or TX) pin on the receiver to an RX pin on the ESP32 (e.g., GPIO 16 or GPIO 17).
   - This signal pin will carry SBUS data to the ESP32, allowing it to receive control inputs from the

transmitter.

Step 2: Flash the ESP32 with ExpressLRS Firmware (Optional)

To use ExpressLRS on the ESP32, you may need to flash firmware if you're using a specific setup for long-range communications, such as telemetry or bidirectional data. This step is optional but recommended for advanced configurations.

1. Download the ExpressLRS Configurator from the ExpressLRS GitHub.
2. Flash the ESP32 with ExpressLRS firmware through the configurator, selecting the proper options for your hardware.

Step 3: Code to Read SBUS Data on ESP32

To read the SBUS data, you'll need code to decode SBUS, which is a serial protocol commonly used for RC systems.

Example Code to Read SBUS Data:

Here's a simplified code to start reading SBUS signals on the ESP32. You may need to install a library like SBUS library for Arduino IDE to decode the signals correctly.

Install the SBUS Library (in Arduino IDE, go to Sketch > Include Library > Manage Libraries, then search for "SBUS").

Code Example:

```
#include <SBUS.h>

SBUS sbus(Serial2);
```

```
int16_t channels[16];
bool failsafe;
bool lostFrame;

void setup() {
  Serial.begin(115200);
  Serial2.begin(100000, SERIAL_8E1, 16, -1);
  sbus.begin();
}

void loop() {
  if (sbus.read(&channels[0], &failsafe, &lostFrame)) {
    Serial.print("Channel 1: ");
    Serial.println(channels[0]);
  }
  delay(20);
}
```

Step 4: Bind the Receiver to the Transmitter

1. Put the Receiver into Binding Mode.

2. Initiate Binding Mode on the Transmitter (e.g., Jumper T14 Macarone).

3. Once bound, the receiver should automatically connect to the transmitter on each power-up.

Step 5: Testing

1. Power on the ESP32 and the transmitter.

2. Open the Serial Monitor in the Arduino IDE to view channel values and test control inputs.

This setup should give you complete control inputs from the transmitter to the ESP32.

ESP32 PlatformIO Code with BAYCK ELRS Receiver

Instructions for Connecting BAYCK ELRS Receiver to ESP32 Using PlatformIO

Step 1: Create a New PlatformIO Project

1. Open VS Code and go to the PlatformIO Home.

2. Create a New Project.

   - Board: Select your specific ESP32 model.

   - Framework: Choose Arduino.

3. Click Finish to create the project.

Step 2: Add the SBUS Library

Add the SBUS library to your platformio.ini dependencies:

```
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = arduino
lib_deps =
    bolderflight/SBUS@^1.0.0  ; Or latest version
monitor_speed = 115200
```

Step 3: ESP32 Code to Read SBUS Data

Use the following code in your src/main.cpp file:

```
#include <Arduino.h>
#include <SBUS.h>

#define SBUS_RX_PIN 16

SBUS sbus(Serial2);

int16_t channels[16];
bool failsafe;
bool lostFrame;

void setup() {
    Serial.begin(115200);
    Serial2.begin(100000, SERIAL_8E1, SBUS_RX_PIN, -1);
    sbus.begin();
}

void loop() {
    if (sbus.read(&channels[0], &failsafe, &lostFrame)) {
        Serial.print("Channel 1: ");
        Serial.println(channels[0]);

        if (failsafe) {
```

```
            Serial.println("Failsafe activated!");
        }

        if (lostFrame) {
            Serial.println("Lost Frame detected!");
        }
    }
    delay(20);
}
```

Step 4: Build and Upload the Code

1. Connect the ESP32 to your computer.
2. In PlatformIO, click Build to compile the code.
3. After a successful build, click Upload to flash it to the ESP32.
4. Open Serial Monitor to view the channel values.

This setup allows you to receive control inputs from the transmitter on the ESP32 using PlatformIO.